

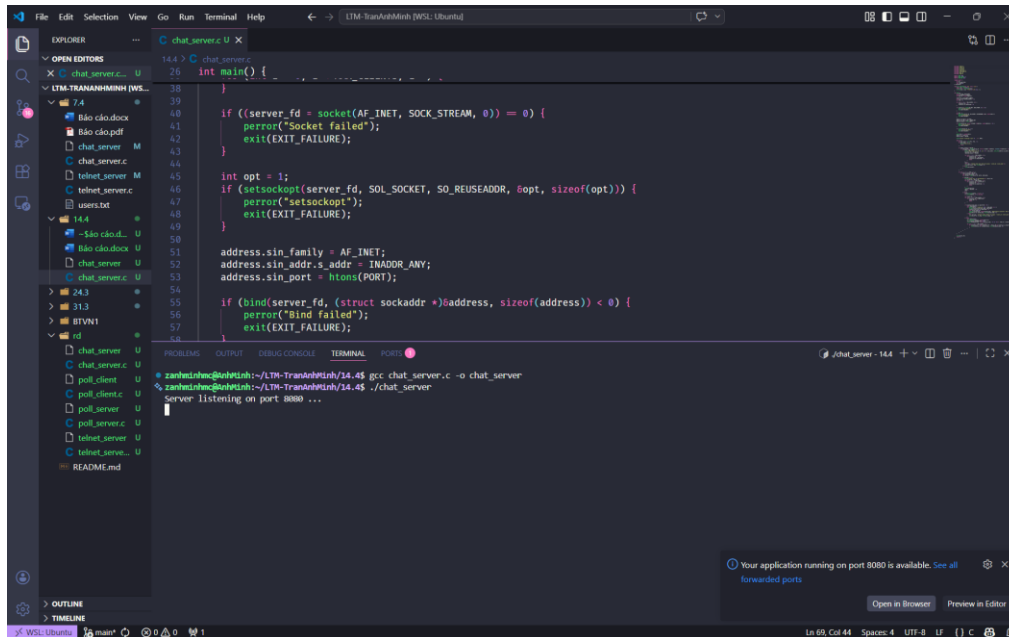
ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO BÀI TẬP
LẬP TRÌNH MẠNG

Trần Anh Minh - 20235382

HÀ NỘI, 04/2026

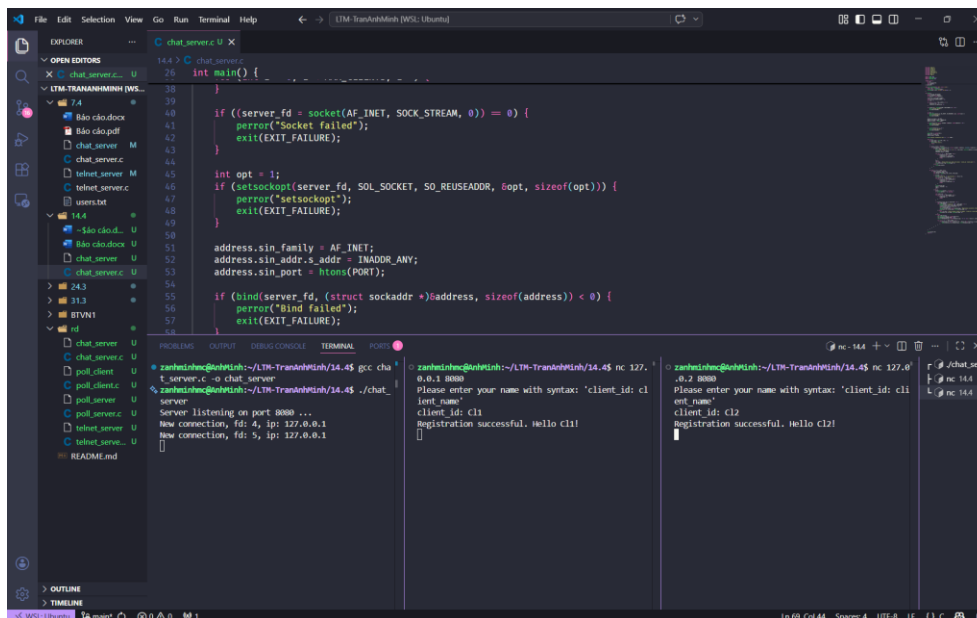
Chat server



The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project structure. The main editor window shows the code for `chat_server.c`. The code includes headers, defines `PORT` as 8080, and implements a `main` function that sets up a socket, binds it to `0.0.0.0:8080`, and enters a loop to accept connections. The terminal at the bottom shows the command `gcc chat_server.c -o chat_server` and the output `Server listening on port 8080 ...`. A notification at the bottom right states "Your application running on port 8080 is available."

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/socket.h>
6  #include <netinet/in.h>
7  #include <arpa/inet.h>
8  #include <string.h>
9  #include <errno.h>
10
11  #define PORT 8080
12
13  int main() {
14
15      if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
16          perror("Socket failed");
17          exit(EXIT_FAILURE);
18      }
19
20      int opt = 1;
21      if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))) {
22          perror("setsockopt");
23          exit(EXIT_FAILURE);
24      }
25
26      address.sin_family = AF_INET;
27      address.sin_addr.s_addr = INADDR_ANY;
28      address.sin_port = htons(PORT);
29
30      if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
31          perror("Bind failed");
32          exit(EXIT_FAILURE);
33      }
34
35      printf("Server listening on port %d ...", PORT);
36
37      while (1) {
38          struct sockaddr_in client_addr;
39          socklen_t client_addr_len = sizeof(client_addr);
40          int new_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_addr_len);
41          if (new_fd < 0) {
42              perror("accept");
43              continue;
44          }
45          printf("New connection, fd: %d, ip: %s\n", new_fd, inet_ntoa(client_addr.sin_addr));
46
47          // Handle the client connection
48      }
49
50      return 0;
51 }
```

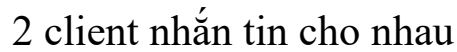
Compiles chương trình và chạy server



The screenshot shows the Visual Studio Code editor with the file explorer on the left. The main editor window shows the code for `chat_server.c`. The terminal at the bottom shows the command `gcc chat_server.c -o chat_server` and the output `Server listening on port 8080 ...`. Two terminal windows are open, showing the execution of the chat server. The first terminal window shows the command `nc 127.0.0.1 8080` and the output `Please enter your name with syntax: 'client_id: client_name'`. The second terminal window shows the command `nc 127.0.0.1 8080` and the output `Please enter your name with syntax: 'client_id: client_name'`. The server output shows `New connection, fd: 4, ip: 127.0.0.1` and `New connection, fd: 5, ip: 127.0.0.1`.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/types.h>
5  #include <sys/socket.h>
6  #include <netinet/in.h>
7  #include <arpa/inet.h>
8  #include <string.h>
9  #include <errno.h>
10
11  #define PORT 8080
12
13  int main() {
14
15      if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
16          perror("Socket failed");
17          exit(EXIT_FAILURE);
18      }
19
20      int opt = 1;
21      if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))) {
22          perror("setsockopt");
23          exit(EXIT_FAILURE);
24      }
25
26      address.sin_family = AF_INET;
27      address.sin_addr.s_addr = INADDR_ANY;
28      address.sin_port = htons(PORT);
29
30      if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
31          perror("Bind failed");
32          exit(EXIT_FAILURE);
33      }
34
35      printf("Server listening on port %d ...", PORT);
36
37      while (1) {
38          struct sockaddr_in client_addr;
39          socklen_t client_addr_len = sizeof(client_addr);
40          int new_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_addr_len);
41          if (new_fd < 0) {
42              perror("accept");
43              continue;
44          }
45          printf("New connection, fd: %d, ip: %s\n", new_fd, inet_ntoa(client_addr.sin_addr));
46
47          // Handle the client connection
48      }
49
50      return 0;
51 }
```

Tạo 2 client C11 và C12



The image shows a Visual Studio Code editor window with the following components:

- Explorer:** Displays the file structure of the project, including folders like `Báo cáo.docx`, `Báo cáo.pdf`, and `chat_server`, and files like `chat_server.c`, `telnet_server.c`, `telnet_server.h`, `users.txt`, and `README.md`.
- Editor:** Shows the source code for `telnet_server.c`. The code implements a Telnet server using `poll()` for non-blocking I/O. It includes headers for `stdio.h`, `stdlib.h`, `string.h`, `unistd.h`, `sys/types.h`, `sys/socket.h`, `sys/time.h`, `sys/uio.h`, and `sys/poll.h`. The `main` function sets up a server socket, listens for connections, and handles incoming data using `poll()` and `recv()`. It reads usernames and passwords from `users.txt` and authenticates users. The server runs on port 8080.
- Terminal:** Shows the execution of the program. The command `gcc telnet_server.c -o telnet_server` is run, followed by `./telnet_server`. The output indicates that the Telnet server is running on port 8080 using `poll()`.
- Output Panel:** Displays the output of the terminal command, showing the server's status and the port it is listening on.

Compile file và khởi chạy server

